# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

| | | | |
|---|---|---|---|
| **Course Code:** | CSA06 | **Course Title:** | Design and Analysis of Algorithms |
| **CO Assessed:** | CO2 – Manipulation (BL1, BL2, BL3) | **Assessment:** | Assessment Tool 3 – Debugging & Optimisation Task |
| **Weightage:** | 10% of CIA | **Total Marks:** | 100 |
| **CO2 Statement:** | *Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)* | | |
| **Student Name:** | GANESH M | **Reg. No.:** | 192524034 |
| **Section / Batch:** | B | **Date:** | 27-07-2026 |

**Instructions to Students**

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

| Question Type | Focus Area | Questions |
|---|---|---|
| **Debugging & Optimisation Task** | Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques | Q1 |

## SECTION A – DEBUGGING & OPTIMISATION TASK

**Code of Conduct**

*I GANESH M / 192524034___(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

## RUBRICS FOR CALCULATION

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Problem Identification | 20% | Accurately identifies all errors and root causes with clear understanding | Identifies most errors with minor gaps | Identifies some errors but misses key issues | Unable to identify errors correctly |
| Debugging | 25% | Applies systematic debugging techniques effectively to resolve issues | Uses appropriate debugging methods with minor errors | Limited debugging approach; requires guidance | Unable to debug or resolve issues |
| Optimization | 20% | Optimizes code by significantly improving time complexity using efficient algorithms | Improves time complexity with minor gaps in efficiency | Limited improvement in time complexity | No improvement; inefficient approach retained |
| Analysis | 20% | Demonstrates strong logical reasoning and clear justification of solutions | Shows reasonable analysis with some justification | Limited analysis; weak reasoning | No analytical reasoning demonstrated |
| Code Quality | 15% | Produces clean, well-structured code with proper comments and documentation | Code is mostly clear with minor documentation gaps | Code lacks structure and proper documentation | Poorly written code with no documentation |

## Marks Evaluation:

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Problem Identification | 20% | | | | |
| Debugging | 25% | | | | |
| Optimization | 20% | | | | |
| Analysis | 20% | | | | |
| Code Quality | 15% | | | | |
| **Total Marks (100):** | | | | | |

| Faculty In-charge | Course Coordinator | Head of Department |
|---|---|---|
| Name: | Name: | Name: |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

**NAME: GANESH M**

**REG NO:192524034**

**COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS**

**COURSE CODE: CSA0603**

QUESTION:

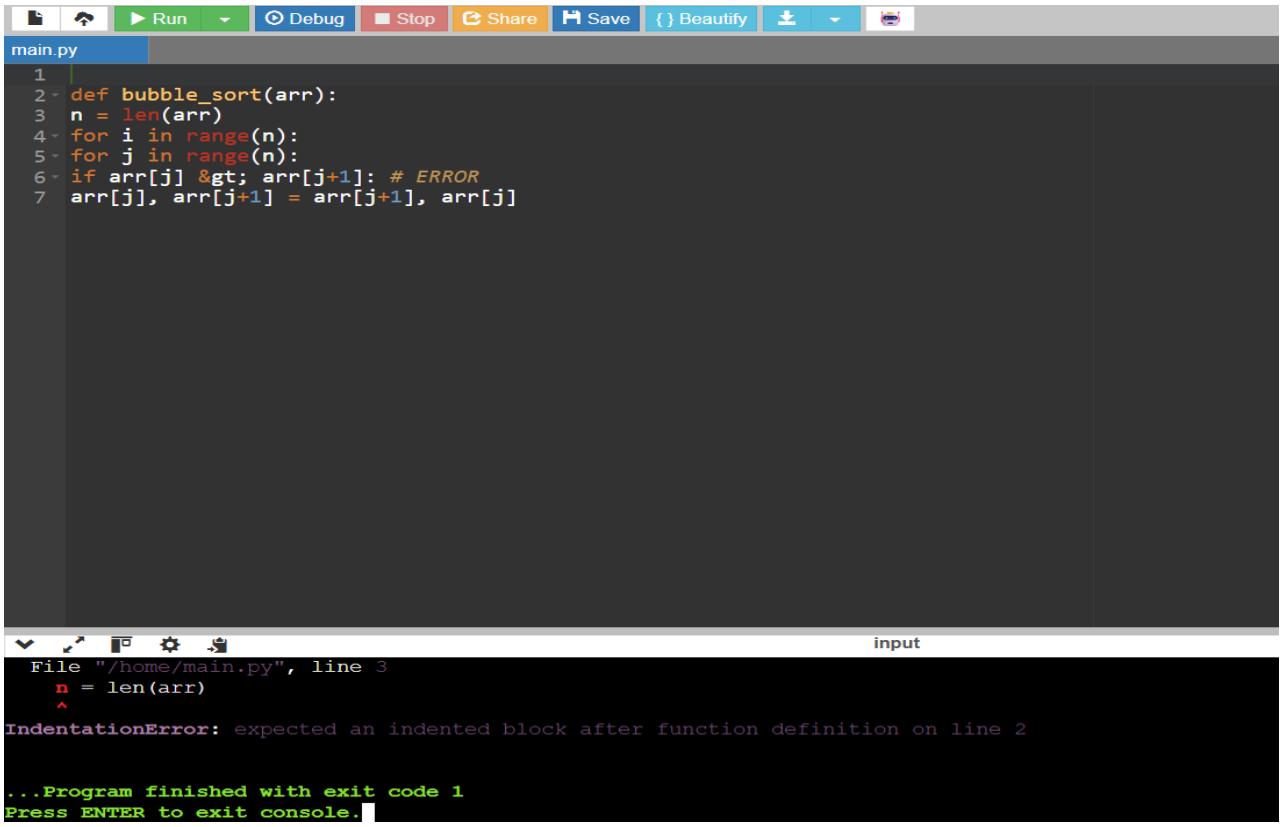
Q13. Debugging Bubble Sort (Incorrect Loop Bounds)

The given Bubble Sort implementation fails to sort correctly due to improper loop bounds. Identify the error in iteration limits and explain its effect. Debug the code step-by-step to ensure proper comparisons. Optimize by reducing redundant passes. Analyze time complexity and provide a corrected version.

```
def bubble_sort(arr):
n = len(arr)
for i in range(n):
for j in range(n):
if arr[j] &gt; arr[j+1]: # ERROR
arr[j], arr[j+1] = arr[j+1], arr[j]
```

**Debugging Bubble Sort**

**Step 1: Given Code (With Error)**




**Identify the Error**

Step 2: Identify the Error
The inner loop is written as:

for j in range(n):

When j becomes n-1, the program tries to access:

arr[j+1]

which becomes:

arr[n]

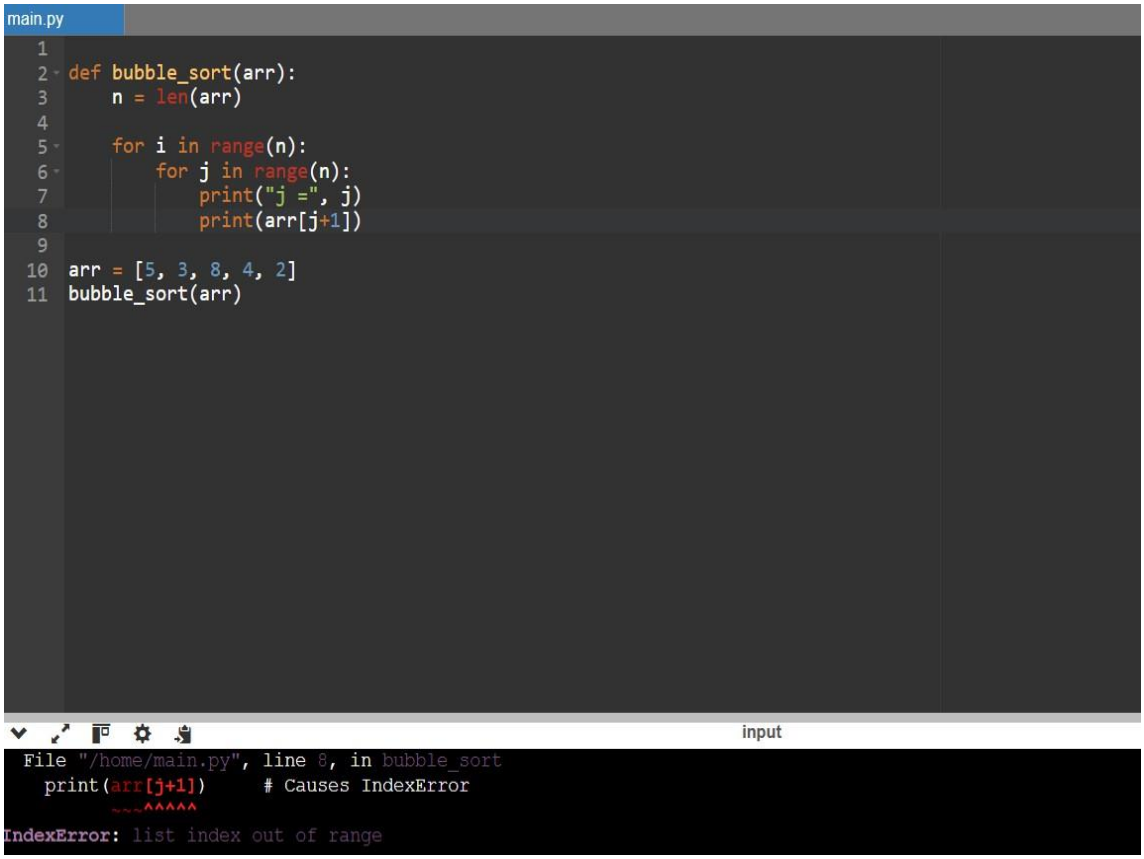
Since the last valid index of the array is n-1, arr[n] does not exist.

**Effect of the Error**

- The program terminates with an error.
- Python displays:

IndexError: list index out of range

The sorting process cannot continue because the program crashes.




**Step 3: Debugging the Code (Fix the Index Error)**

Change:

for j in range(n):

to

for j in range(n-1):

**Explanation**

The last comparison is between:

arr[n-2] and arr[n-1]

Therefore, the loop should stop at n-2, which is achieved by

using: range(n-1)

This prevents the program from accessing an invalid index.

**Step 4: Second Debugging (Remove Redundant Comparisons)**

After every pass, the largest element reaches the

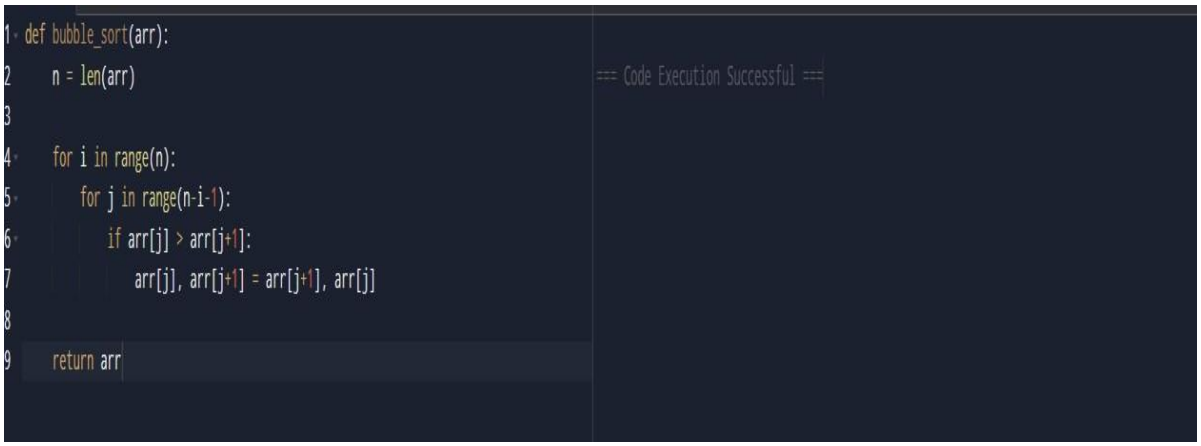
end. So there is no need to compare it again.

**Before**

for j in range(n-1):

**After**

for j in range(n-i-1):




**Step 5: Final Optimized Bubble Sort**

If no swapping happens during a pass, the array is already

sorted. Use a **swapped** flag.

Correct debugging bubble sort code:

```
def
   bubble_sort(arr):
   n = len(arr)
   for i in range(n - 1):
      for j in range(n - i -
         1): if arr[j] > arr[j
         + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
arr = [64, 34, 25, 12, 22, 11, 90]
bubble_sort(arr)
print("Sorted Array:")
```

```
print(arr)
```

```python
def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

arr = [64, 34, 25, 12, 22, 11, 90]

bubble_sort(arr)

print("Sorted Array:")
print(arr)
```

```
input
Sorted Array:
[11, 12, 22, 25, 34, 64, 90]


...Program finished with exit code 0
Press ENTER to exit console.
```

Bubble Sort Algorithm Algorithm:

1. Start the process.
2. Read the array and find the number of elements n.
3. Repeat for each pass from i = 0 to n-1:
    - Set swapped = False.
    - Compare each pair of adjacent elements from j = 0 to n-i-2.
    - If arr[j] > arr[j+1], swap the two elements and set swapped = True.
4. If no swapping occurs during a pass (swapped = False), stop the algorithm because the array is already sorted.
5. Repeat until all passes are completed or the array becomes sorted.
6. Display the sorted array.
7. Stop.

**Bubble Sort – Step-by-Step Manual Analysis .**

Given Array:
[64, 34, 25, 12, 22, 11, 90]

Number of Elements (n): 7

**Pass 1 (i = 0)**

During the first pass, Bubble Sort compares adjacent elements from the beginning of the array. The number 64 is compared with 34, 25, 12, 22, and 11, and since 64 is greater than each of them, it is swapped every time. Finally, 64 is compared with 90, and no swap occurs because 64 is smaller than
90. At the end of the first pass, the array becomes [34, 25, 12, 22, 11, 64, 90]. The largest element, 90, is now in its correct position at the end of the array.

**Pass 2 (i = 1)**

In the second pass, the last element (90) is already sorted, so it is not included in the comparisons. The algorithm compares 34 with 25, 12, 22, and 11, swapping whenever necessary. When 34 is compared with 64, no swap occurs because 34 is smaller. After completing this pass, the array becomes [25, 12, 22, 11, 34, 64, 90]. Now both 64 and 90 are in their correct positions.

**Pass 3 (i = 2)**

The third pass ignores the last two sorted elements (64 and 90). The algorithm compares 25 with 12, 22, and 11, performing swaps where required. When 25 is compared with 34, no swap is needed. At the end of this pass, the array becomes [12, 22, 11, 25, 34, 64, 90]. The element 34 has reached its correct position.

**Pass 4 (i = 3)**

During the fourth pass, only the unsorted portion of the array is processed. The algorithm compares 12 with 22, and since they are already in the correct order, no swap occurs. Next, 22 is compared with 11, and they are swapped. Finally, 22 is compared with 25, and no swap is required. The array after this pass becomes [12, 11, 22, 25, 34, 64, 90].

**Pass 5 (i = 4)**

Only the first two comparisons are required in this pass. The algorithm compares 12 with 11, and since 12 is greater, they are swapped. Then 12 is compared with 22, and no swap occurs. The array now becomes [11, 12, 22, 25, 34, 64, 90].

**Pass 6 (i = 5)**

In the final pass, only one comparison is performed between 11 and 12. Since the elements are already in ascending order, no swap is needed. The array remains [11, 12, 22, 25, 34, 64, 90].

**Final Sorted Array**

**Original Array:**
[64, 34, 25, 12, 22, 11, 90]

**Sorted Array:**
[11, 12, 22, 25, 34, 64, 90]

**Time Complexity :**

Best Case (Already Sorted) - O(n)

Average Case - $O(n^2)$ Time

Worst Case - $O(n^2)$

**Space Complexity**

Space complexity - O(1)

Step 7: Summary of Debugging

| Step | Error | Fix |
|---|---|---|
| 1 | for j in range(n) | Changed to range(n-1) to avoid IndexError. |
| 2 | Unnecessary comparisons | Changed to range(n-i-1) to skip sorted elements. |
| 3 | Extra passes after sorting | Added swapped flag to stop early if the array is already sorted. |

**EXPLANATION OF THE CORRECT CODE AND DEBUGGING:**

- Corrected Bubble Sort program sorts the array by repeatedly comparing two adjacent elements. If the left element is greater than the right element, they are swapped. After each pass, the largest unsorted element moves to its correct position at the end of the array. The inner loop is reduced using range(n-i-1) because the last i elements are already sorted. A swapped flag is used to check whether any exchange occurs during a pass. If no swapping occurs, the algorithm stops early since the array is already sorted, improving efficiency.

- The original code contained an error in the inner loop (range(n)), which caused the program to access arr [j+1] when j was the last index, resulting in an **IndexError**. This was fixed by changing the loop to range(n-1), preventing out-of-range access. Next, the algorithm was optimized by changing the loop to range(n-i-1) to avoid unnecessary comparisons with already sorted elements. Finally, a swapped flag was added to terminate the algorithm early if no swaps occur during a pass, making the Bubble Sort implementation more efficient while producing the correct sorted output.